FulfillOps — environment setup and submission guide (9.5/10 edition)

1. Development platform (one tested configuration per team)

Use **Linux, macOS, or Windows 11 with WSL2 Ubuntu** for the container/Kubernetes tasks. Recommended minimum laptop resources: 16 GB RAM (8 GB might work with a reduced cluster), 4 CPU cores, and 25 GB free disk. Internet is needed for initial tool and container-image downloads. You may work in a local VM if your university restricts Docker Desktop. Have a GitHub account; request access to a private repo with the instructor before the deadline.

Install Git, Docker Engine/Desktop with Docker Compose v2, kubectl, kind, Python 3.11+ for the contract checker, and your chosen backend runtime (Node.js 22 LTS OR Python 3.11+ are recommended). A local PostgreSQL 16 container is recommended. Argo CD will be installed inside kind. The Argo CD CLI is optional; kubectl is sufficient for most tasks.

Official installation references:

Git: https://git-scm.com/downloadsX

Docker Desktop: https://docs.docker.com/desktop/X

Docker Compose: https://docs.docker.com/compose/install/X

Python: https://www.python.org/downloads/X

Node.js: https://nodejs.org/en/downloadX

kubectl: https://kubernetes.io/docs/tasks/tools/X

kind: https://kind.sigs.k8s.io/docs/user/quick-start/X

Argo CD: https://argo-cd.readthedocs.io/en/stable/getting_started/X

Windows note: Run kind, Docker CLI, Bash and Kubernetes commands in the **same WSL2 Ubuntu shell** where your repository is located. Enable the Docker Desktop WSL integration. Do not mix Windows paths with WSL paths in bind mounts. Restart WSL after enabling integration if needed.

2. Verify the prerequisites *before* coding

```
git --version
docker version
docker compose version
kubectl version --client
kind version
python3 --version
# or: python --version
# your chosen runtime:
node --version
```

Verify Docker daemon access and image download:

```
docker run --rm hello-world
```

If the Python checker is on Windows PowerShell, invoke py -3 fulfillops_public_checker.py; on Linux/macOS use python3 if python is not available.

3. Create your own repository and source files

The student pack **has no application code**. Create a fresh empty Git repository and your own project from scratch. Recommended checkpoints:

```
mkdir fulfillops && cd fulfillops
git init -b main
# Create your source tree and README yourself.
# Copy the downloaded API_CONTRACT_EN.md and checker into a docs/ or assessment/ folder if useful.
```

Choose a data store and queue strategy, then write your first architecture decision record before implementing the API. Check in source and dependency lockfiles. Commit incremental work on feature branches and open PRs for review. Never commit .env with secrets.

4. Create the local stack and assess the contract

You implement compose.yaml, migrations, seed scripts, API and worker. The expected interface is http://localhost:8080. The following is an **outcome**, not a supplied command guaranteed to work before you implement the stack:

```
docker compose up -d --build
docker compose ps
curl http://localhost:8080/health/ready
python3 fulfillops_public_checker.py --base-url http://localhost:8080
# WARNING: destructive stock test; use only a disposable test dataset:
python3 fulfillops_public_checker.py --base-url http://localhost:8080 --concurrency --stress
```

The checker creates orders and consumes inventory; the --stress option additionally exercises eight simultaneous same-key requests and duplicate SKU-line aggregation. Use a dedicated testing database, seeded with 7 KIT-100, 12 PART-200 and 5 PACK-300. A re-run may skip creation if the stock is already exhausted. Your safe **test-only** reset script must never operate on a production database.

5. Kubernetes and Argo CD setup

```
kind create cluster --name fulfillops --wait 120s
kubectl cluster-info --context kind-fulfillops
kubectl get nodes
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-repo-server --timeout=300s
kubectl -n argocd rollout status deploy/argocd-application-controller --timeout=300s || true
```

For newer Argo CD releases the Application Controller may run as a StatefulSet; check kubectl -n argocd get deploy,sts before interpreting the optional rollout line. Pin a specific Argo CD release for any claimed reproducible setup and document the version used. The stable manifest reference above is for initial teaching setup, not an immutable production install.

Build and push the images to a registry accessible by kind **or** load them into the cluster and document the choice:

```
kind load docker-image your-image:your-commit-sha --name fulfillops
```

Build the Argo CD Application and Kubernetes manifests yourself. The repo must be accessible to Argo CD (public repository or appropriately configured read-only access for a private repository). **Do not paste a GitHub token into a manifest or commit credentials.** Use an Argo CD repository Secret stored/managed out of Git.

For local API access:

kubectl -n fulfillops port-forward svc/fulfillops-api 8080:80

The namespace/service names above are suggested conventions; adapt and document them if you choose different names.

6. Observability and reliability demo

At minimum produce: time-stamped structured logs, healthy/unhealthy readiness evidence, /metrics, a clearly described SLO calculation, and an incident timeline. Simulate a failure only in your isolated test cluster. Before and after the fault, show whether committed orders survive worker/container restarts and what the user sees. No outage simulation against production or another team's services.

7. Troubleshooting quick reference

Docker daemon not reachable — First checks: Docker Desktop running; WSL integration; docker info

npm ci or pip install fails — First checks: runtime version, lockfile, registry connectivity

Checker cannot connect — First checks: docker compose ps, logs, port 8080, readiness

API responds 503 — First checks: database connectivity and migrations, not simply process liveness

kind cluster creation fails — First checks: Docker RAM/CPU, stale cluster, networking, node image download

ImagePullBackOff — First checks: tag/digest, registry access, kind load, imagePullPolicy

Argo CD cannot sync — First checks: repository access, incorrect path, missing CRDs, namespace

PersistentVolumeClaim Pending — First checks: storage class, kind default storage provisioner

Secret leak in logs — First checks: remove secret from logs and rotate exposed value immediately

Race checker oversells — First checks: database transaction isolation, conditional updates, unique idempotency constraint

8. Submission checklist

☐ Git repository and instructor access verified

☐ Source files, lockfiles, migrations, tests, Compose and Kubernetes configuration committed

☐ README.md has exact fresh-install and teardown commands

☐ **15** public checker checks pass with --concurrency --stress (on a fresh isolated database)

☐ PR pipeline with red-to-green evidence, SBOM and scan artifact

☐ Docker build and Compose evidence, persistent order after restart

☐ kind/Argo CD deployment evidence, drift reconciliation, two API and two worker pods

☐ two fault injections, worker retry/dead-letter evidence, 30-second rollout probe, recovery, metrics, SLO and postmortem

☐ max eight-page report and 12-15-minute demo

☐ individual contributions + assistance disclosures

☐ no secrets, production data, personal data or instructor files in student ZIP

These exercises include **six supervised classroom hours** divided into two 3-hour blocks. Use the in-class time for architecture review, first concurrency test, worker/recovery rehearsal and deployment evidence. A team should budget approximately **34–49 additional team-hours**.

Suggested final submission deadline: **Sunday 15 November 2026, 23:59 Europe/Paris**. Confirm with instructor.

9. Classroom checkpoint and advanced preflight

The instructor will ask for two checkpoints during the six classroom hours; no code solution is supplied.

Checkpoint A (end of first 3 h): demonstrate one safe stock-reservation transaction, replay semantics, architecture/ADR and independently written test.

Checkpoint B (end of second 3 h): demonstrate worker restart recovery, an enforced CI gate, a controlled deployment diagnostic and a completed evidence checklist.

To run the final API checker against the **fresh disposable test stack**:

```
python3 fulfillops_public_checker.py --base-url http://localhost:8080 --concurrency --stress
```

When --stress is skipped due to a depleted PACK-300 inventory, **reset only the isolated test DB** and rerun. The final grading screenshot must show the full checks with no skips. The public checker does not test Kubernetes or kill workers; teams must deliver separate reproducible demonstration scripts for these operational requirements.

For GitOps, do not put database credentials in the repository. For worker crash tests, deliberately stop only an isolated classroom worker; use independent probes to confirm that a committed order survives and is processed once. Any destructive step must have a documented cleanup/rollback.

30-second rolling-update evidence (from an isolated local port-forward)

With the Kubernetes API Service port-forwarded to localhost:8080, start the supplied standard-library probe *before* initiating a known-good rolling update. It records the HTTP status, UTC time and response duration on every sample. The script exits with a nonzero code if any probe fails.

```
python3 fulfillops_rollout_probe.py --base-url http://localhost:8080 \
  --path /health/ready --duration 30 --interval 0.25 \
  --output docs/rolling_update_probe.csv
```

Store the CSV under docs/ and record the Deployment/Argo CD revisions, time window and what you changed. For the **broken** release drill, preserve a separate failure log and use a Git revert to restore the known-good state. Never run the probe against another team's instance or a real production service.